



UNIVERSITAS
INDONESIA

CEP-CCIT

FAKULTAS TEKNIK

**Implementation of Python and SQL Programming for a Digital
Diary**

Group 7

Names : Hilwa Annisa
: Rizky Nur Aziz

Class : 3CS1

Faculty : Ivan Firdaus, S.T.
Tri Agus Riyadi, S.Kom., M.Kom

CEP-CCIT

FAKULTAS TEKNIK UNIVERSITAS INDONESIA

2025

PROJECT INFORMATION

Implementation of Python and SQL Programming for a Digital Diary

Project Title : Implementation of Python and SQL Programming for a Digital Diary

Batch Code : 3CS1

Start Date : September 12, 2025

End Date : September 22, 2025

Name of Faculty : 1. Ivan Firdaus, S.T.
2. Tri Agus Riyadi, S.Kom., M.Kom

Names of Developer:

1. Hilwa Annisa
2. Rizky Nur Aziz

Date of Submission: September 24, 2025

ACKNOWLEDGEMENT

First of all, we would like to express our greatest gratitude to God Almighty for His endless blessings and guidance, which have enabled us to continue our studies at CCIT FTUI and successfully complete this project without major obstacles. We are truly grateful for the opportunity and strength given to us throughout the process of working on this paper.

We also sincerely express our appreciation to our respected lecturers, Mr. Ivan Firdaus, S.T. and Mr. Tri Agus Riyadi, S.Kom., M.Kom., for their valuable guidance, constructive feedback, and encouragement. Their support and recommendations have been very meaningful for us in finishing this project. We would also like to acknowledge the dedication and hard work of our group members, Rizky Nur Aziz and Hilwa Annisa, who have collaborated and contributed greatly to the success of this OOP CRUD Python MySQL project.

This paper aims to implement Object Oriented Programming (OOP) in developing a CRUD system using Python and MySQL as its database. We realize that this paper is far from perfect, therefore we sincerely apologize for any mistakes or shortcomings in the writing. We are very open to receiving constructive suggestions and criticisms, and we hope that despite its limitations, this work can still provide useful insights and benefits to readers.

Depok, 24th Sept 2025

Authors

SYSTEM ANALYSIS

1. Description

The digital diary system is a straightforward Python application that integrates MySQL for data storage. The purpose of this program is to help users safely and systematically document their daily experiences or activities.

2. Purpose

- a. Providing digital means for recording daily diaries.
- b. Simplifying note management thru CRUD (Create, Read, Update, Delete) operations.
- c. Providing automatically saved data based on the input date.

3. Main feature

- a. User registration and login.
- b. Adding a new diary entry, mood, achievement.
- c. Viewing the list of notes that have already been created.
- d. Changing the content of the note.
- e. Deleting records that are no longer needed.

4. Benefits of the System:

- Notes are safer because they are stored in a database.
- Facilitates searching for notes by date and title.
- Each user has full control over their own records.

5. System Limitations:

- The interface is command-line based (CLI) and does not yet support a graphical display.
- Does not support file uploads (images, audio, video), only text.
- The system runs on a local server with MySQL (XAMPP).

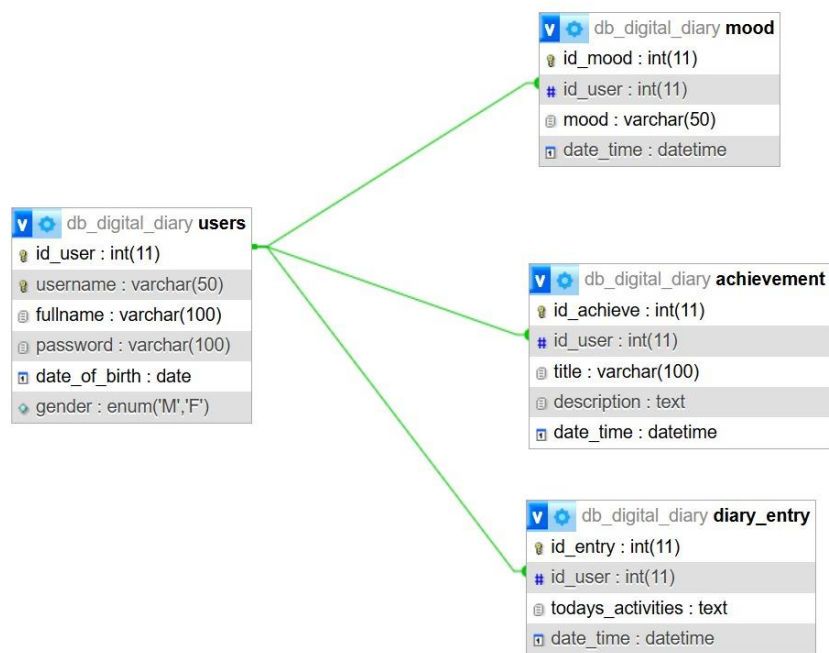
BACKGROUND

In today's digital age, recording daily activities no longer requires a notebook. Digital diary applications make it easy, secure, and efficient to record everything. Digital diary systems allow users to record their daily experiences, thoughts, and achievements in a structured manner, making it easy to store, retrieve, and manage personal information anytime, anywhere.

To build such a system, the combination of the Python programming language and the MySQL database offers an effective solution. Python provides simplicity and flexibility in application development, while MySQL ensures that data can be stored, updated, and accessed securely. By applying the principles of Object Oriented Programming (OOP), the system can be designed in a structured and maintainable manner, making the development process more organized and easier to develop.

Therefore, this project aims to develop a Digital Diary application that implements CRUD (Create, Read, Update, Delete) functions using Python and MySQL with an OOP approach. Through this project, users are expected to have a reliable tool for managing their daily notes digitally, while readers can gain a deeper understanding of how OOP concepts and database integration are applied in real world application development.

Entity Relationship Diagram (ERD)



Description

1. User table attribute

- id_user (PK) : primary key for unique user identification.
- username
- fullname
- password
- date_of_birth
- gender

Entity Relationship Diagram

2. Mood table attribute

- id_mood (PK) : The main key to any mood record.
- id_user (FK) : The key to any mood entry is linking the mood to a specific user.
- mood
- date_time

3. Achievement table attribute

- id_achieve (PK) : The main key to achievement.
- id_user (FK) : users who record achievements.
- description
- date_time

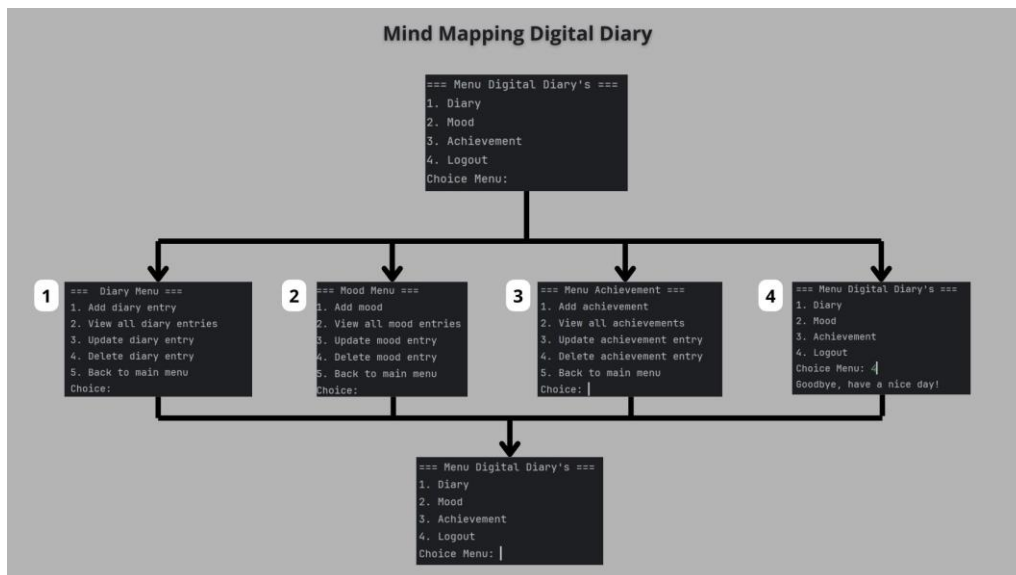
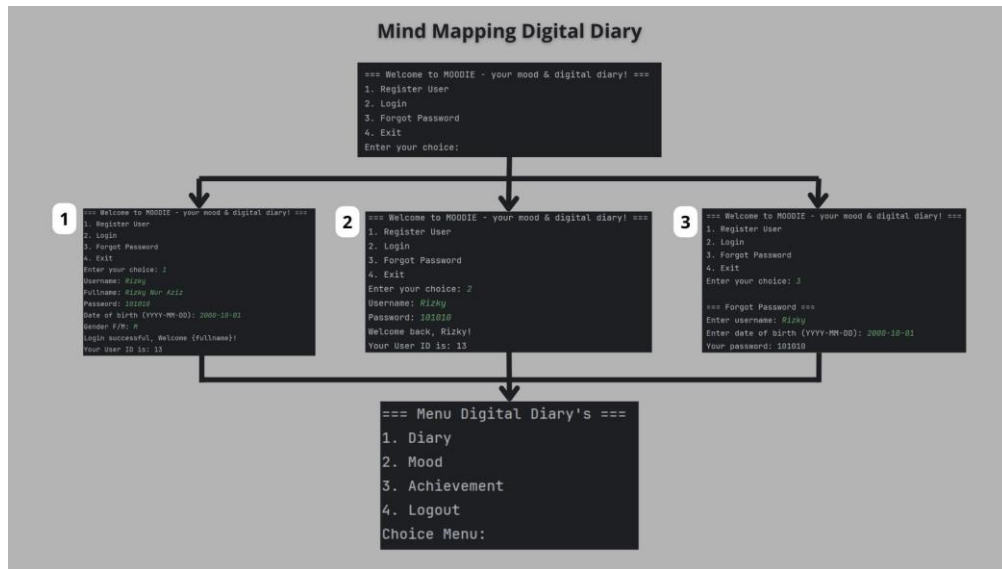
4. Diary entry attribute

- id_entry (PK) : primary key for the diary entry.
- user_id (FK) : the user who wrote the note.
- Today's activities
- date_time

5. Relationship

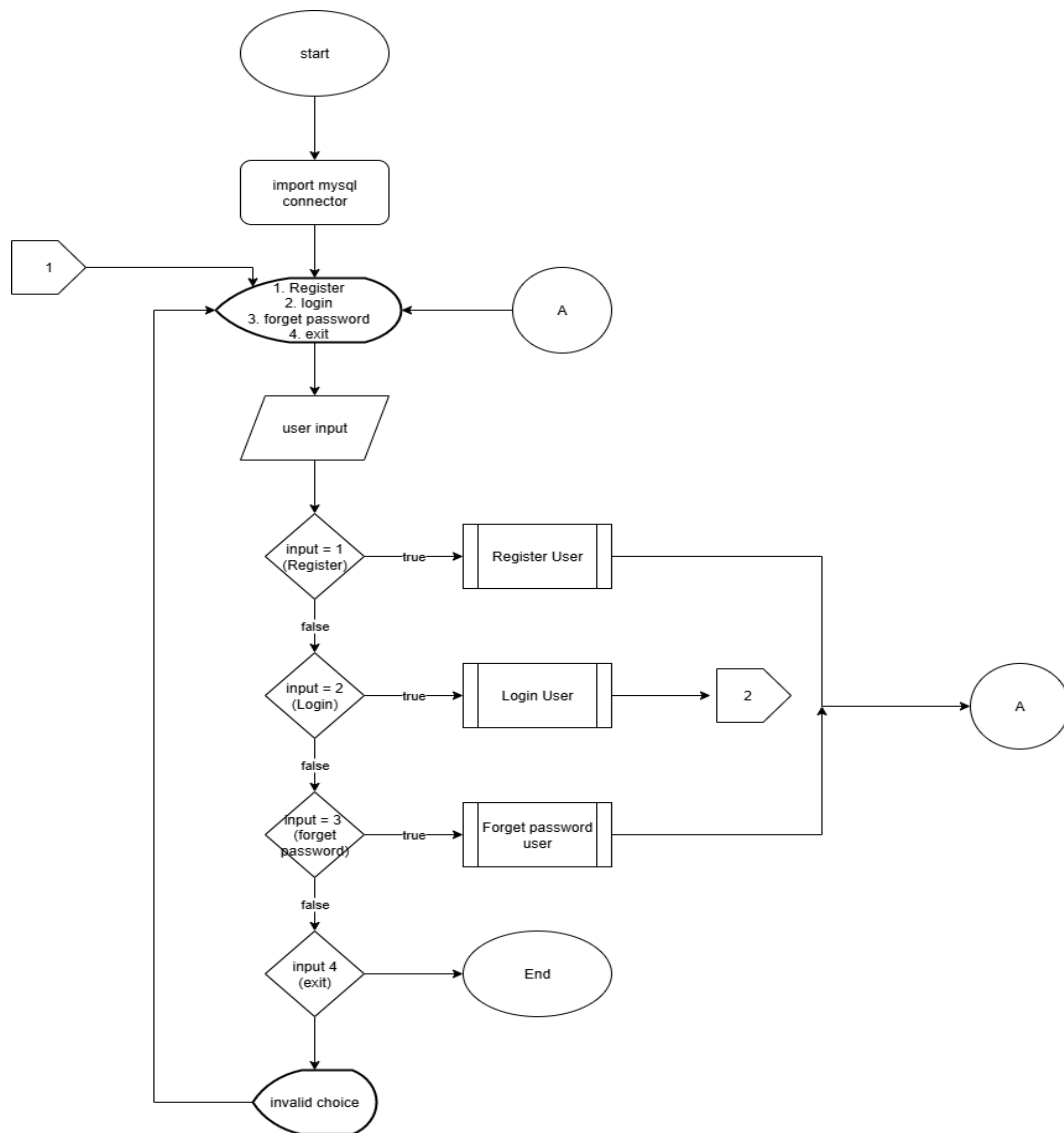
- The "users" entity becomes the center of the relationship.
- users (1) = (N) mood
- users (1) = (N) achievement
- users (1) = (N) diary_entry

MIND MAPPING



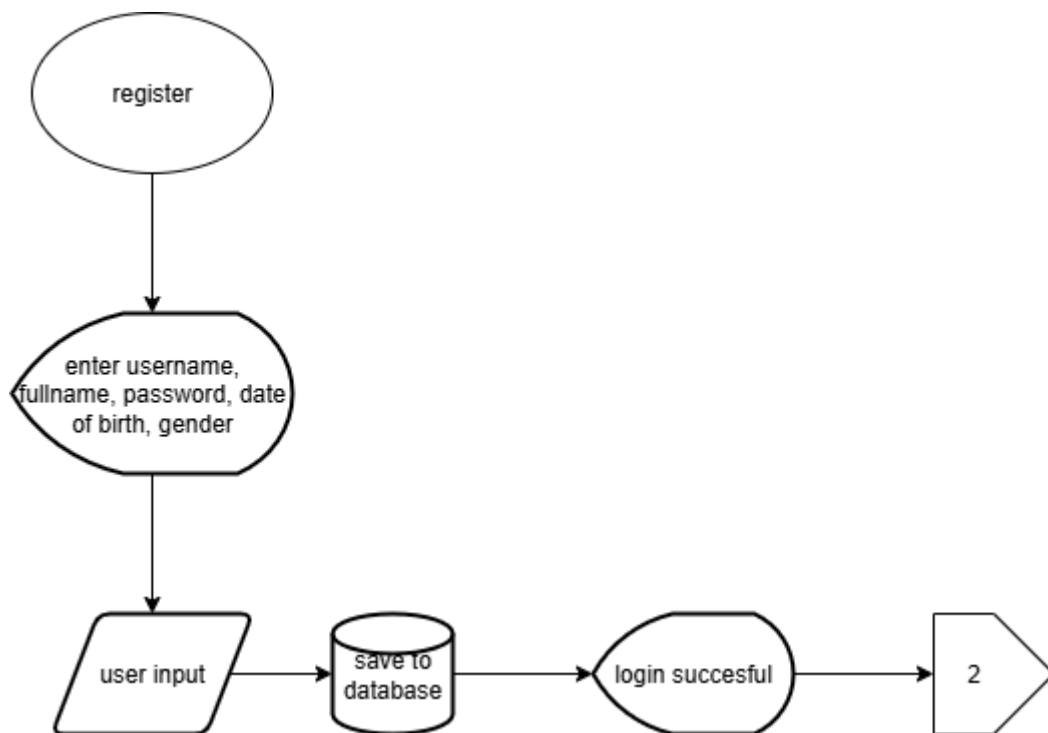
FLOWCHART

Start



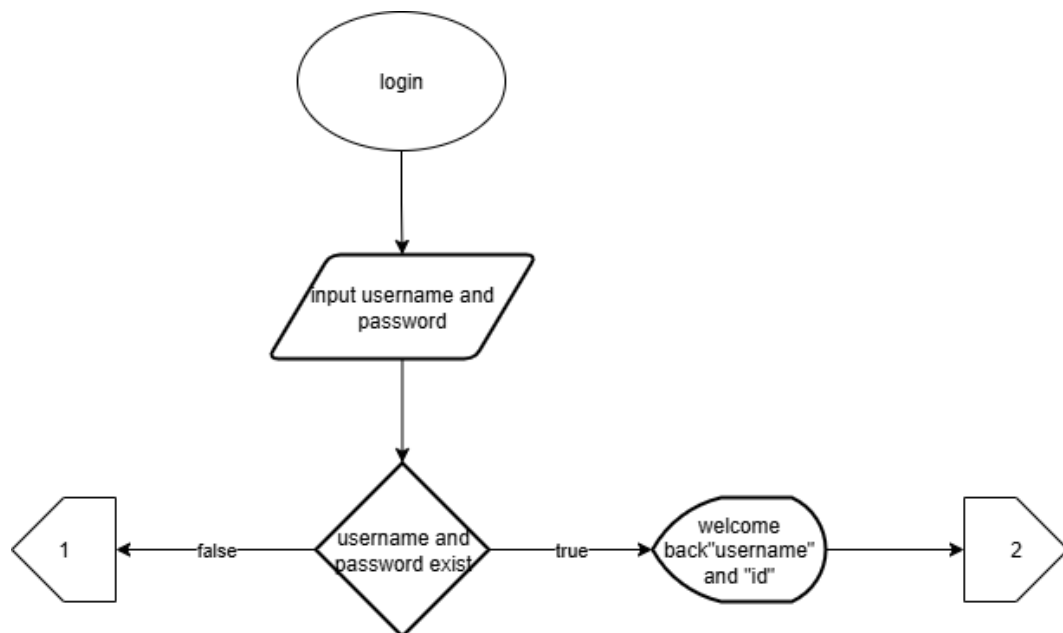
FLOWCHART

Register

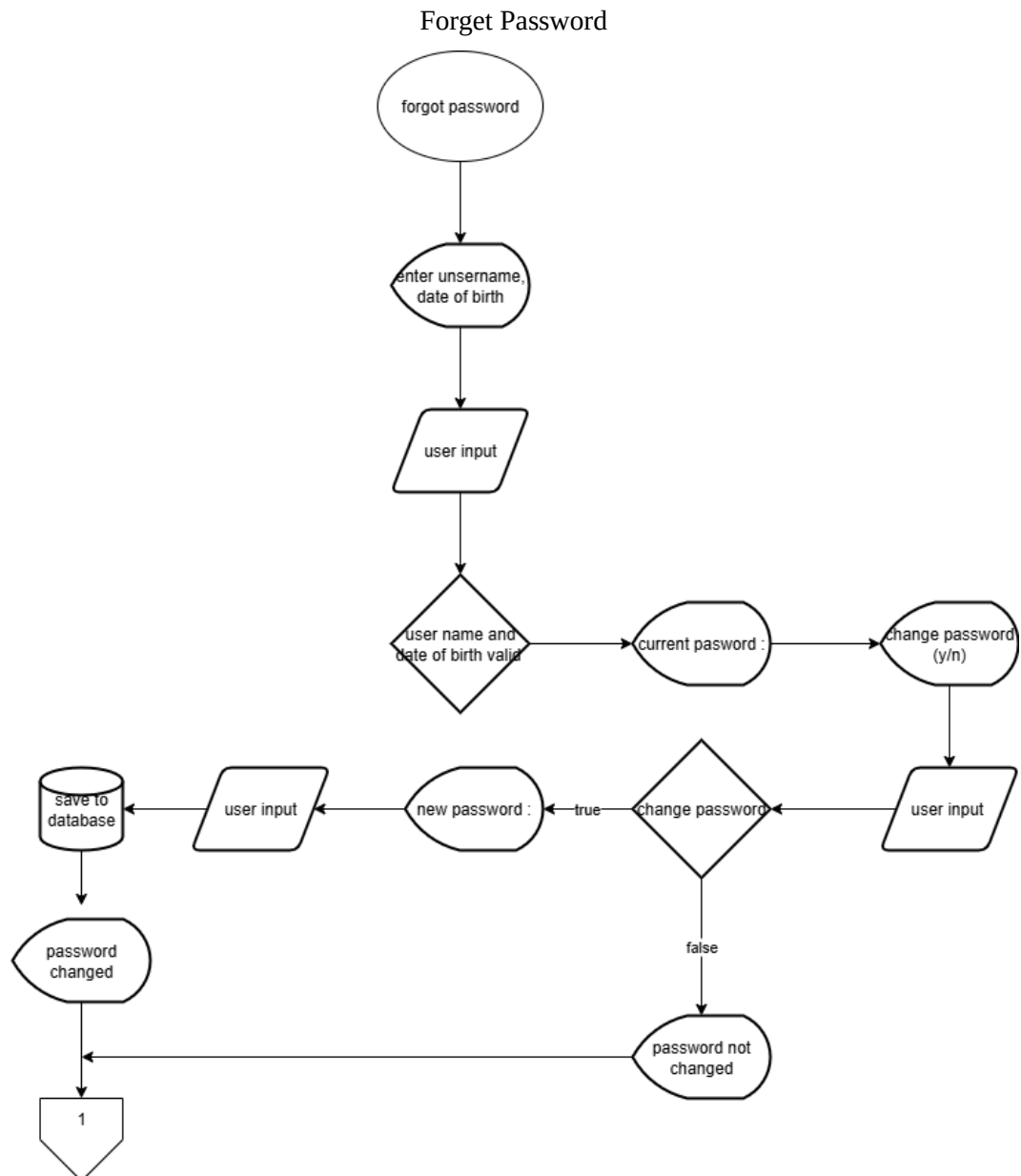


FLOWCHART

Login

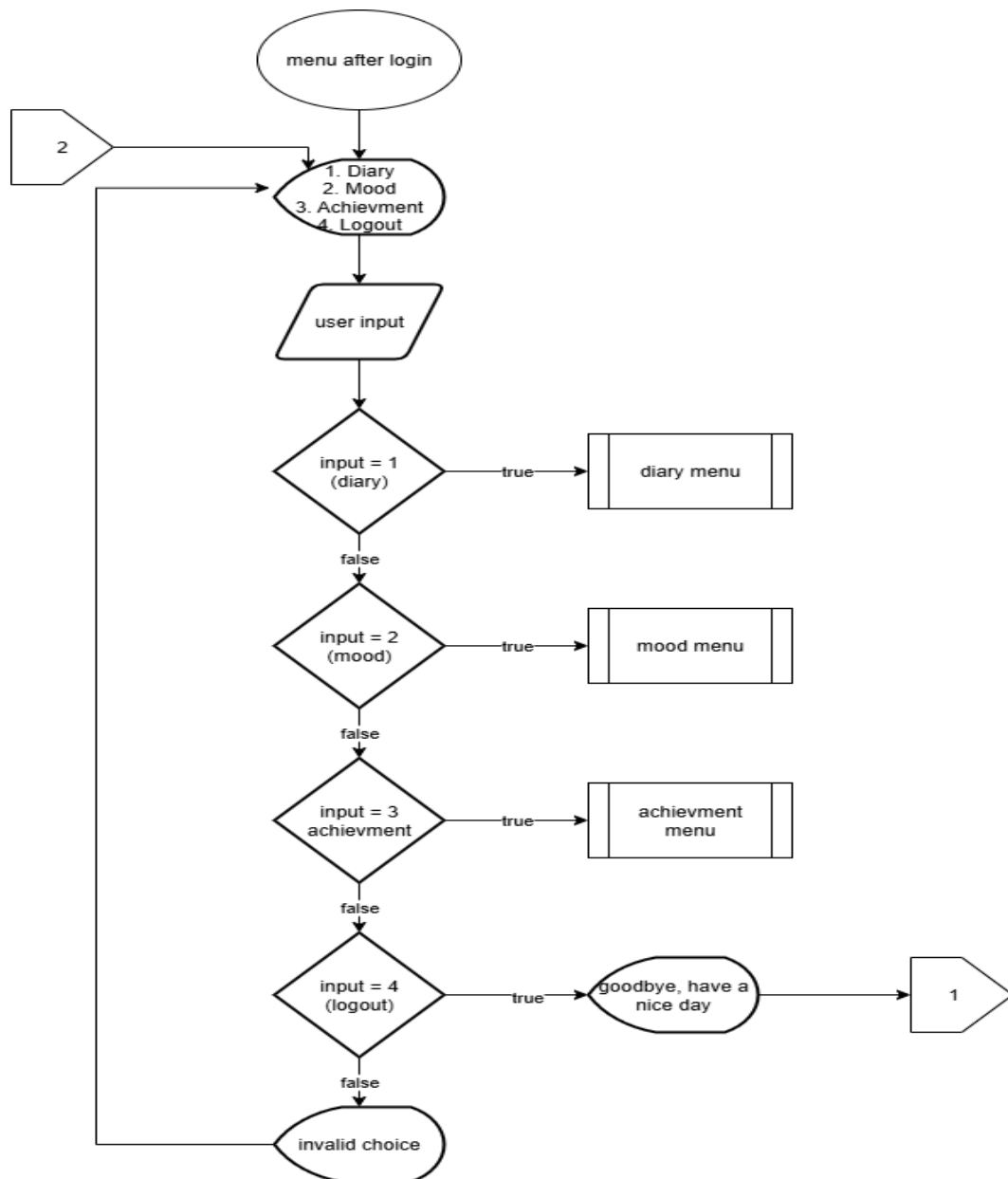


FLOWCHART



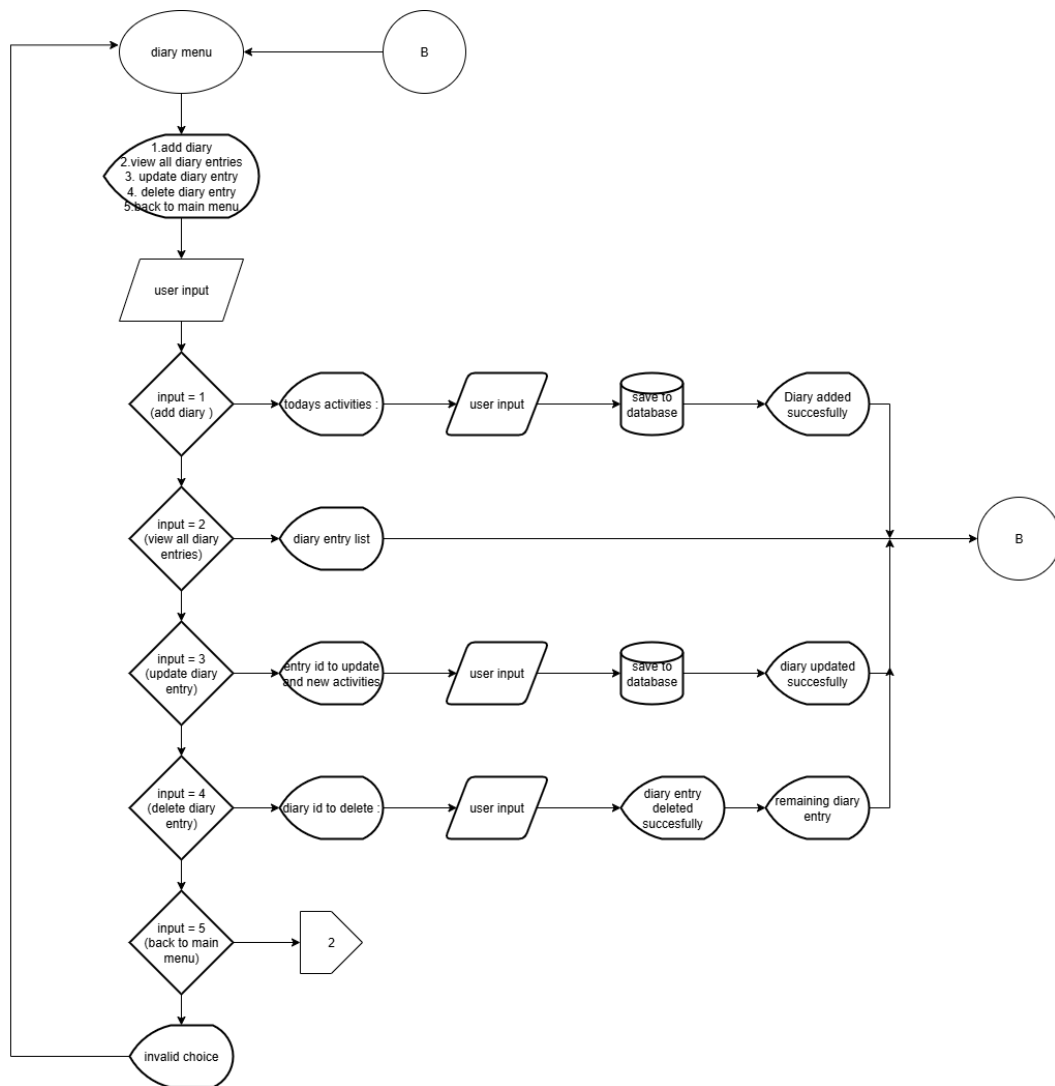
FLOWCHART

Menu after login



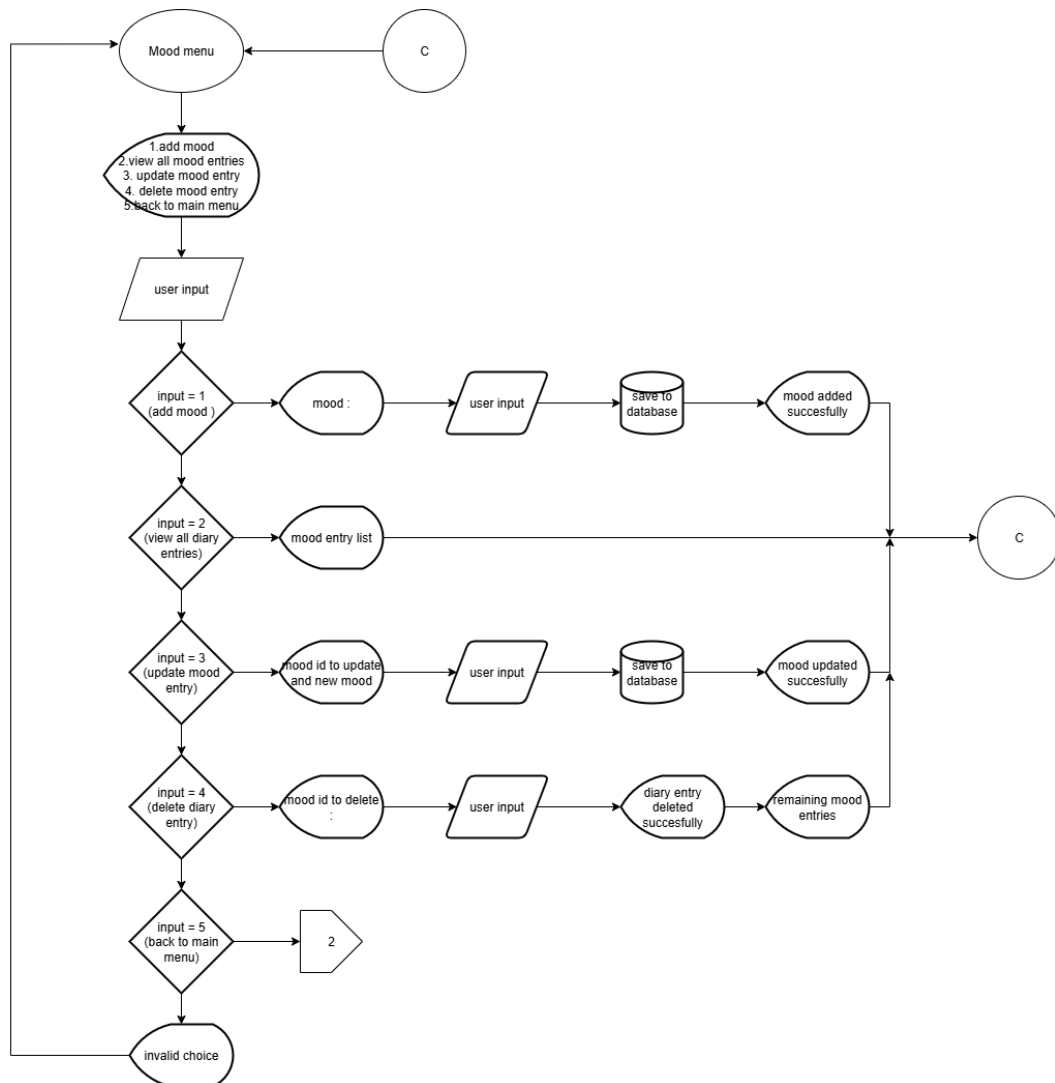
FLOWCHART

Diary menu



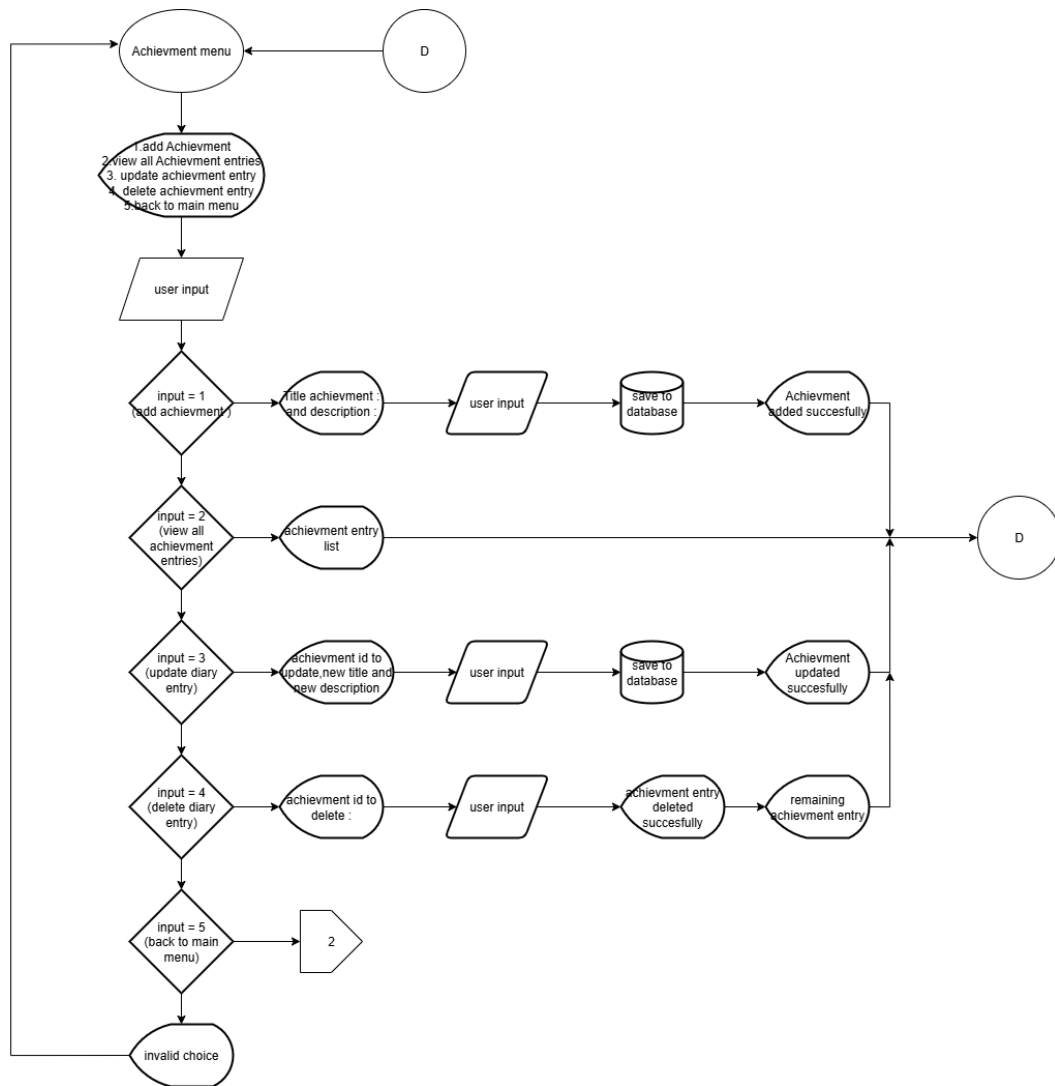
FLOWCHART

Mood menu



FLOWCHART

Achievment menu



DATABASE DIGITAL DIARY

1. Database of user

✔Displaying rows 0 - 4 (total 5, Search performed in 0.0003 seconds.)

SELECT * FROM `users`

Profile

Edit in box

Change

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows:

Search in this table

Sort by key:

There isn't any

Extra options

⬆

⬇

⬇

⬆

id_user

username

full name

password

date_of_birth

gender

⬆

⬇

⬇

⬆

Change

Translation

happy

1

Officer

Raya Luna

12345

2003-09-12

F

⬆

⬇

⬇

⬆

Change

Translation

happy

2

Star

Latitude of Light

abcd123

2002-05-20

M

⬆

⬇

⬇

⬆

Change

Translation

happy

3

Ayu

Ayu Putri

221133

2004-11-01

F

⬆

⬇

⬇

⬆

Change

Translation

happy

12

yusuf

Yusuf yoga

565656

2008-02-01

F

⬆

⬇

⬇

⬆

Change

Translation

happy

13

Rizky

Rizky Nur Aziz

101010

2000-10-01

M

⬆

⬇

⬇

⬆

Select All

With options:

Change

Translation

happy

Export

Show all

Number of rows: 25

Filter rows:

Search in this table

Sort by key:

There isn't any

2. Database of diary entry

✔ Displaying rows 0 - 6 (7 total, Search performed in 0.0004 seconds.)

SELECT * FROM `diary\_entry`

☐ Profile ☐ Edit in box ☐ Change ☐ Explain SQL ☐ Create PHP code ☐ Refresh

☐ Show all | Number of rows: 25 | Filter rows: Search in this table | Sort by key: There isn't any

Extra options

				id_entry	id_user	todays_activities	date_time
☐			happy	1	1	this	2025-09-20 15:40:48
☐			happy	3	2	Ui Olim supporters practice with friends	2025-09-20 15:40:48
☐			happy	5	1	yoi	2025-09-20 15:40:48
☐			happy	15	12	today was fun	2025-09-24 14:21:50
☐			happy	17	1	dawssdawdadaw	2025-09-24 14:44:21
☐			happy	19	13	Today was so exciting. I just came back from my fr...	2025-09-25 15:03:42
☐			happy	20	13	huft... I'm so tired after taking the exam	2025-09-25 15:32:09

⬆ ☐ Select All With options: Change Translation happy Export

☐ Show all | Number of rows: 25 | Filter rows: Search in this table | Sort by key: There isn't any

DATABSE DIGITAL DIARY

3. Database of mood

✔ Displaying rows 0 - 4 (total 5, Search performed in 0.0004 seconds.)

`SELECT * FROM `mood``

☐ Profile [Edit in box](#) [Change](#) [Explain SQL](#) [Create PHP code](#) [Refresh](#)

☐ Show all | Number of rows: 25 | Filter rows: Search in this table | Sort by key: There isn't any

Extra options

			<code>id_mood</code>	<code>id_user</code>	<code>mood</code>	<code>date_time</code>
☐	Change	Translation	happy	1	1 like	2025-09-20 16:22:18
☐	Change	Translation	happy	2	1 Tired	2025-09-20 16:22:18
☐	Change	Translation	happy	3	2 Happy	2025-09-20 16:22:18
☐	Change	Translation	happy	4	13 Happy	2025-09-24 10:49:29
☐	Change	Translation	happy	5	13 fun!	2025-09-24 11:51:03

☐ Select All | With options: [Change](#) [Translation](#) [happy](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search in this table | Sort by key: There isn't any

4. Database of achievement

✔ Displaying rows 0 - 2 (total 3, Search performed in 0.0009 seconds.)

`SELECT * FROM `achievement``

☐ Profile [Edit in box](#) [Change](#) [Explain SQL](#) [Create PHP code](#) [Refresh](#)

☐ Show all | Number of rows: 25 | Filter rows: Search in this table | Sort by key: There isn't any

Extra options

			<code>id_achieve</code>	<code>id_user</code>	<code>title</code>	<code>description</code>	<code>date_time</code>
☐	Change	Translation	happy	1	1 Completing a Python Project	Successfully created a digital diary CRUD application with...	2025-09-20 16:25:51
☐	Change	Translation	happy	2	1 1st Place in Coding Competition	Faculty level competition in the web development category	2025-09-20 16:25:51
☐	Change	Translation	happy	3	2 First Time Writing a Regular Diary	Consistent writing for 7 consecutive days	2025-09-20 16:25:51

☐ Select All | With options: [Change](#) [Translation](#) [happy](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search in this table | Sort by key: There isn't any

FUNCTION PYTHON

1. Function to connect python program to MySQL

```
import mysql.connector
from datetime import datetime

# ===== Database =====
class Database: 15 usages
    def __init__(self):
        self.db = mysql.connector.connect(
            host="localhost",
            user="root",
            passwd="",
            database="db_digital_diary"
        )
        self.cursor = self.db.cursor(dictionary=True)
```

```
    def execute(self, sql, val=None): 11 usages
        self.cursor.execute(sql, val)
        self.db.commit()

    def fetchone(self): 2 usages
        return self.cursor.fetchone()

    def fetchall(self): 3 usages
        return self.cursor.fetchall()

    def close(self): 17 usages
        self.cursor.close()
        self.db.close()
```

2. Class of user for add new user

```
# ===== User =====
class User: 3 usages
    def __init__(self, username=None, fullname=None, password=None, date_of_birth=None, gender=None):
        self.id_user = None
        self.username = username
        self.fullname = fullname
        self.password = password
        self.date_of_birth = date_of_birth
        self.gender = gender
```

FUNCTION PYTHON

3. function to display for registered user

```
def register(self):
    usage
    db = Database()
    sql = """INSERT INTO users (username, fullname, password, date_of_birth, gender)
            VALUES (%s, %s, %s, %s, %s)"""
    val = (self.username, self.fullname, self.password, self.date_of_birth, self.gender)
    db.execute(sql, val)
    print(f"User {self.username} registered successfully!")
    db.close()
```

4. Function to display login user

```
def login_user(self, username, password):
    usage
    db = Database()
    db.cursor.execute(operation="SELECT * FROM users WHERE username = %s", params=(username,))
    user = db.fetchone()

    if not user:
        print("User not found")
        db.close()
        return False

    if user["password"] == password:
        self.id_user = user["id_user"]
        self.username = user["username"]
        self.fullname = user["fullname"]
        print(f"Welcome back, {self.fullname}! (ID: {self.id_user})")
        db.close()
        return True
    else:
        print("Invalid username or password")
        db.close()
        return False
```

5. Function to display forget password user

```
def forget_password(self):
    usage
    db = Database()
    print("\n== Forget Password ==")
    username = input("Enter username: ")
    dob = input("Enter date of birth (YYYY-MM-DD): ")

    sql = "SELECT id_user, password FROM users WHERE username = %s AND date_of_birth = %s"
    db.cursor.execute(sql, params=(username, dob))
    result = db.fetchone()

    if result:
        print(f"Your current password: {result['password']}")
        choice = input("Do you want to update your password? (y/n): ").lower()
        if choice == 'y':
            new_pass = input("Enter new password: ")
            sql_update = "UPDATE users SET password = %s WHERE id_user = %s"
            db.execute(sql_update, val=(new_pass, result["id_user"]))
            print("Password updated successfully!")
        else:
            print("Password not changed.")
    else:
        print("No matching user found.")

    db.close()
```

FUNCTION PYTHON

6. Class of entry for add diary entry

```
# ===== Entry (Parent) =====  
class Entry: 3 usages  
    def __init__(self, user_id):  
        self.user_id = user_id  
        self.date_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")  
  
    def add_entry(self):  
        raise NotImplementedError
```

7. Function of add entry

```
# ===== Diary =====  
class Diary(Entry): 2 usages  
    def __init__(self, user_id, activities=None):  
        super().__init__(user_id)  
        self.activities = activities  
  
    def add_entry(self): 1 usage  
        db = Database()  
        sql = "INSERT INTO diary_entry (id_user, today's_activities, date_time) VALUES (%s, %s, %s)"  
        val = (self.user_id, self.activities, self.date_time)  
        db.execute(sql, val)  
        print("Diary entry added successfully!")  
        db.close()
```

FUNCTION PYTHON

8. Function to view all diary user

```
def view_all(self): 1 usage
    db = Database()
    sql = "SELECT id_entry, todays_activities, date_time FROM diary_entry WHERE id_user = %s"
    db.cursor.execute(sql, params: (self.user_id,))
    entries = db.fetchall()
    if entries:
        print("\n=== Your Diary Entries ===")
        for e in entries:
            print(f"ID: {e['id_entry']} | Activities: {e['todays_activities']} | Date: {e['date_time']}")
    else:
        print("No diary entries found.")
    db.close()
```

9. Function to update diary user

```
def update(self, entry_id, new_activities): 1 usage
    db = Database()
    sql = "UPDATE diary_entry SET todays_activities=%s WHERE id_entry=%s AND id_user=%s"
    db.execute(sql, val: (new_activities, entry_id, self.user_id))
    if db.cursor.rowcount > 0:
        print("Diary updated successfully!")
    else:
        print("No diary entry found.")
    db.close()
```

10. Function to delete diary user

```
def delete(self, entry_id): 1 usage
    db = Database()
    sql = "DELETE FROM diary_entry WHERE id_entry=%s AND id_user=%s"
    db.execute(sql, val: (entry_id, self.user_id))
    if db.cursor.rowcount > 0:
        print("Diary entry deleted successfully!")
    else:
        print("No diary entry found.")
    db.close()
```

11. Class of mood entry

```
# ===== Mood =====
class Mood(Entry): 2 usages
    def __init__(self, user_id, mood=None):
        super().__init__(user_id)
        self.mood = mood
```

12. Function to add mood entry

```
def add_entry(self): 1 usage
    db = Database()
    sql = "INSERT INTO mood (id_user, mood, date_time) VALUES (%s, %s, %s)"
    val = (self.user_id, self.mood, self.date_time)
    db.execute(sql, val)
    print("Mood added successfully!")
    db.close()
```

FUNCTION PYTHON

13. Function to view all mood

```
def view_all(self): 1 usage
    db = Database()
    sql = "SELECT id_mood, mood, date_time FROM mood WHERE id_user = %s"
    db.cursor.execute(sql, params: (self.user_id,))
    moods = db.fetchall()
    if moods:
        print("\n=== Your Mood Entries ===")
        for m in moods:
            print(f"ID: {m['id_mood']} | Mood: {m['mood']} | Date: {m['date_time']}")
    else:
        print("No mood entries found.")
    db.close()
```

14. Function to update mood

```
def update(self, mood_id, new_mood): 1 usage
    db = Database()
    sql = "UPDATE mood SET mood=%s WHERE id_mood=%s AND id_user=%s"
    db.execute(sql, val: (new_mood, mood_id, self.user_id))
    if db.cursor.rowcount > 0:
        print("Mood updated successfully!")
    else:
        print("No mood entry found.")
    db.close()
```

15. Function to delete mood

```
def delete(self, mood_id): 1 usage
    db = Database()
    sql = "DELETE FROM mood WHERE id_mood=%s AND id_user=%s"
    db.execute(sql, val: (mood_id, self.user_id))
    if db.cursor.rowcount > 0:
        print("Mood entry deleted successfully!")
    else:
        print("No mood entry found.")
    db.close()
```

FUNCTION PYTHON

16. Class of achievement entry

```
# ===== Achievement =====
class Achievement(Entry): 2 usages
    def __init__(self, user_id, title=None, description=None):
        super().__init__(user_id)
        self.title = title
        self.description = description
```

17. Function to add achievement

```
def add_entry(self): 1 usage
    db = Database()
    sql = "INSERT INTO achievement (id_user, title, description, date_time) VALUES (%s, %s, %s, %s)"
    val = (self.user_id, self.title, self.description, self.date_time)
    db.execute(sql, val)
    print("Achievement added successfully!")
    db.close()
```

18. Function to view all achievement

```
def view_all(self): 1 usage
    db = Database()
    sql = "SELECT id_achieve, title, description, date_time FROM achievement WHERE id_user = %s"
    db.cursor.execute(sql, (self.user_id,))
    results = db.fetchall()
    if results:
        print("\n=== Your Achievements ===")
        for a in results:
            print(f"ID: {a['id_achieve']} | Title: {a['title']} | Description: {a['description']} | Date: {a['date_time']}")
    else:
        print("No achievements found.")
    db.close()
```

19. Function to update achievement

```
def update(self, achieve_id, new_title, new_desc): 1 usage
    db = Database()
    sql = "UPDATE achievement SET title=%s, description=%s WHERE id_achieve=%s AND id_user=%s"
    db.execute(sql, (val, (new_title, new_desc, achieve_id, self.user_id))
    if db.cursor.rowcount > 0:
        print("Achievement updated successfully!")
    else:
        print("No achievement entry found.")
    db.close()
```

20. Function to delete achievement

```
def delete(self, achieve_id): 1 usage
    db = Database()
    sql = "DELETE FROM achievement WHERE id_achieve=%s AND id_user=%s"
    db.execute(sql, (val, (achieve_id, self.user_id))
    if db.cursor.rowcount > 0:
        print("Achievement deleted successfully!")
    else:
        print("No achievement entry found.")
    db.close()
```


FUNCTION PYTHON

20. Function to after login menu user

```
# ===== Menu =====
def after_login_menu(user): 1usage
    while True:
        print("\n== Menu Digital Diary's ==")
        print("1. Diary")
        print("2. Mood")
        print("3. Achievement")
        print("4. Logout")
        choice = input("Choice: ")

        if choice == "1":
            diary_menu(user)
        elif choice == "2":
            mood_menu(user)
        elif choice == "3":
            achievement_menu(user)
        elif choice == "4":
            print("Goodbye, have a nice day!")
            break
        else:
            print("Invalid choice")
```

21. Function to diary menu user

```
def diary_menu(user): 1usage
    diary = Diary(user.id_user)
    while True:
        print("\n== Diary Menu ==")
        print("1. Add Diary")
        print("2. View All")
        print("3. Update")
        print("4. Delete")
        print("5. Back")
        choice = input("Choice: ")
```

```
    if choice == "1":
        activities = input("Activities: ")
        diary = Diary(user.id_user, activities)
        diary.add_entry()
    elif choice == "2":
        diary.view_all()
    elif choice == "3":
        entry_id = input("Entry ID: ")
        new_act = input("New Activities: ")
        diary.update(entry_id, new_act)
    elif choice == "4":
        entry_id = input("Entry ID: ")
        diary.delete(entry_id)
    elif choice == "5":
        break
```

FUNCTION PYTHON

22. Function to mood menu user

```
def mood_menu(user): 1 usage
    mood = Mood(user.id_user)
    while True:
        print("\n=== Mood Menu ===")
        print("1. Add Mood")
        print("2. View All")
        print("3. Update")
        print("4. Delete")
        print("5. Back")
        choice = input("Choice: ")
```

```
    if choice == "1":
        m = input("Mood: ")
        mood = Mood(user.id_user, m)
        mood.add_entry()
    elif choice == "2":
        mood.view_all()
    elif choice == "3":
        mood_id = input("Mood ID: ")
        new_m = input("New Mood: ")
        mood.update(mood_id, new_m)
    elif choice == "4":
        mood_id = input("Mood ID: ")
        mood.delete(mood_id)
    elif choice == "5":
        break
```

23. Function to achievement menu user

```
def achievement_menu(user): 1 usage
    ach = Achievement(user.id_user)
    while True:
        print("\n=== Achievement Menu ===")
        print("1. Add Achievement")
        print("2. View All")
        print("3. Update")
        print("4. Delete")
        print("5. Back")
        choice = input("Choice: ")
```

PROGRAM OUTPUT

```
if choice == "1":
    title = input("Title: ")
    desc = input("Description: ")
    ach = Achievement(user.id_user, title, desc)
    ach.add_entry()
elif choice == "2":
    ach.view_all()
elif choice == "3":
    aid = input("Achievement ID: ")
    new_t = input("New Title: ")
    new_d = input("New Description: ")
    ach.update(aid, new_t, new_d)
elif choice == "4":
    aid = input("Achievement ID: ")
    ach.delete(aid)
elif choice == "5":
    break
```

25. Function to Main Menu page

```
def main(): 1 usage
    while True:
        print("\n== Welcome to MOODIE - your mood & digital diary! ==")
        print("1. Register")
        print("2. Login")
        print("3. Forgot Password")
        print("4. Exit")
        choice = input("Enter choice: ")
```

PROGRAM OUTPUT

```
if choice == "1":
    username = input("Username: ")
    fullname = input("Fullname: ")
    password = input("Password: ")
    dob = input("Date of birth (YYYY-MM-DD): ")
    gender = input("Gender M/F: ")
    user = User(username, fullname, password, dob, gender)
    user.register()

elif choice == "2":
    username = input("Username: ")
    password = input("Password: ")
    user = User()
    if user.login_user(username, password):
        after_login_menu(user)

elif choice == "3":
    user = User()
    user.forgot_password()

elif choice == "4":
    print("Goodbye!")
    break
else:
    print("Invalid choice")
```

```
if __name__ == "__main__":
    main()
```

PROGRAM OUTPUT

1. When the program starts, the user will see the Main Menu with four options: Register User, Login, Forgot Password, and Exit. When the user selects "Register User," the program will prompt them to fill in their username, full name, password, date of birth, and gender. Then, the program will automatically assign a user ID.

```
=== Welcome to MOODIE - your mood & digital diary! ===
1. Register User
2. Login
3. Forgot Password
4. Exit
Enter your choice: 1
Username: Nabila
Fullname: Nabila Ratu
Password: 454545
Date of birth (YYYY-MM-DD): 2003-01-02
Gender F/M: F
Login successful, Welcome {fullname}!
Your User ID is: 15
```

2. If the user chooses to log in, the program will direct them to enter their username and password.

```
=== Welcome to MOODIE - your mood & digital diary! ===
1. Register User
2. Login
3. Forgot Password
4. Exit
Enter your choice: 2
Username: Nabila
Password: 454545
Welcome back, Nabila
Your User ID is: 15
```

PROGRAM OUTPUT

3. , If the user chooses "forgot password," they are asked to enter their username and date of birth for verification. After that, the system displays the user's old password. Next, the user is given the option to update the password or keep the old one. In the example above, the user chose not to change the password, so the program displayed the message "Password not changed."

```
=== Welcome to MOODIE - your mood & digital diary! ===
1. Register User
2. Login
3. Forgot Password
4. Exit
Enter your choice: 3

=== Forgot Password ===
Enter username: yusuf
Enter date of birth (YYYY-MM-DD): 2008-02-01
Your current password: 565656
Do you want to update your password? (y/n): n
Password not changed.
```

4. After the user successfully logs in, the program will display the main menu consisting of 5 options: diary, mood achievement, and logout.

```
=== Menu Digital Diary's ===
1. Diary
2. Mood
3. Achievement
4. Logout
```

PROGRAM OUTPUT

5. If the user selects the diary option, the program will display 5 options as shown below. If the user selects the "Add diary" option, they can add new notes, view all notes, update, or delete existing notes. In the example above, the user selected the "Add diary entry" option and wrote the daily activity "Hi, Rosay was fun." After that, the system provided a notification that the note was successfully added with the message "Diary entry added successfully!"

```
=== Menu Digital Diary's ===
1. Diary
2. Mood
3. Achievement
4. Logout
Choice Menu: 1

=== Diary Menu ===
1. Add diary entry
2. View all diary entries
3. Update diary entry
4. Delete diary entry
5. Back to main menu
Choice: 1
Today's activities: haii, rosay was fun
Diary entry added successfully!
```

6. If the user selects "View all diary entries," the program will display all the notes the user has created, complete with numbers, activity content, and the recording time (DateTime), making it easier for the user to view their daily activity history.

```
=== Diary Menu ===
1. Add diary entry
2. View all diary entries
3. Update diary entry
4. Delete diary entry
5. Back to main menu
Choice: 2

=== Your Diary Entries ===
NO: 1, Activities: hiii, DateTime: 2025-09-20 15:40:48
NO: 5, Activities: yoi, DateTime: 2025-09-20 15:40:48
NO: 17, Activities: dawssdandadaw, DateTime: 2025-09-24 14:44:21
NO: 22, Activities: haii, rosay was fun , DateTime: 2025-09-28 10:15:01
```

PROGRAM OUTPUT

7. If the user selects "Update diary entry," they can update the diary entry by selecting a specific entry ID. The program then saves the changes and displays a notification that the update was successful.

```
=== Diary Menu ===
1. Add diary entry
2. View all diary entries
3. Update diary entry
4. Delete diary entry
5. Back to main menu
Choice: 3
Entry ID to update: 22
New activities: haii, today was fun
Diary updated successfully!
```

8. If the user chooses to delete a diary entry, they can delete a specific entry based on its ID. The program then displays a list of the remaining entries after the deletion is successfully completed.

```
=== Diary Menu ===
1. Add diary entry
2. View all diary entries
3. Update diary entry
4. Delete diary entry
5. Back to main menu
Choice: 4
Entry ID to delete: 5
Diary entry deleted successfully!

=== Your Remaining Diary Entries ===
NO: 1, Activities: hiii, DateTime: 2025-09-20 15:40:48
NO: 17, Activities: dawdssdawdadaw, DateTime: 2025-09-24 14:44:21
NO: 22, Activities: haii, today was fun, DateTime: 2025-09-28 10:15:01
```


PROGRAM OUTPUT

9. If the user chooses to return to the main menu, the program will return to the main menu.

```
=== Mood Menu ===
1. Add mood
2. View all mood entries
3. Update mood entry
4. Delete mood entry
5. Back to main menu
Choice: 5

=== Menu Digital Diary's ===
1. Diary
2. Mood
3. Achievement
4. Logout
Choice Menu:
```

10. If the user selects the mood option, the program will display 5 options as shown below.

```
=== Menu Digital Diary's ===
1. Diary
2. Mood
3. Achievement
4. Logout
Choice Menu: 2

=== Mood Menu ===
1. Add mood
2. View all mood entries
3. Update mood entry
4. Delete mood entry
5. Back to main menu
Choice: |
```

PROGRAM OUTPUT

11. If the user selects "Add mood," they can add their mood, and the program will display a notification that the mood has been successfully added.

```
=== Mood Menu ===
1. Add mood
2. View all mood entries
3. Update mood entry
4. Delete mood entry
5. Back to main menu
Choice: 1
Mood: Sad
Mood added successfully!
```

12. If the user selects "View all moods," the program will display a list of moods the user has recorded, including entry numbers and recording times.

```
=== Mood Menu ===
1. Add mood
2. View all mood entries
3. Update mood entry
4. Delete mood entry
5. Back to main menu
Choice: 2
No: 1, Mood: senang, DateTime: 2025-09-20 16:22:18
No: 2, Mood: Capek, DateTime: 2025-09-20 16:22:18
No: 7, Mood: 2025-09-28 10:22:19, DateTime: None
```

PROGRAM OUTPUT

13. If the user selects "Update mood entry," the user can update their mood based on ID, and then the program displays a notification that the changes have been successfully saved.

```
=== Mood Menu ===
1. Add mood
2. View all mood entries
3. Update mood entry
4. Delete mood entry
5. Back to main menu
Choice: 4
Mood ID to delete: 7
Mood entry deleted successfully!

=== Remaining Mood Entries ===
NO: 1, Mood: happy, DateTime: 2025-09-20 16:22:18
NO: 2, Mood: Capek, DateTime: 2025-09-20 16:22:18
```

14. If the user chooses to delete a mood entry, they can delete the mood based on ID. The program then displays the list of remaining moods after the deletion is successful.

```
=== Mood Menu ===
1. Add mood
2. View all mood entries
3. Update mood entry
4. Delete mood entry
5. Back to main menu
Choice: 4
Mood ID to delete: 7
Mood entry deleted successfully!

=== Remaining Mood Entries ===
NO: 1, Mood: happy, DateTime: 2025-09-20 16:22:18
NO: 2, Mood: Capek, DateTime: 2025-09-20 16:22:18
```

PROGRAM OUTPUT

15. If the user chooses to return to the main menu, the program will return to the main menu.

```
=== Mood Menu ===
1. Add mood
2. View all mood entries
3. Update mood entry
4. Delete mood entry
5. Back to main menu
Choice: 5

=== Menu Digital Diary's ===
1. Diary
2. Mood
3. Achievement
4. Logout
Choice Menu:
```

16. If the user selects the achievement option, the program will display 5 options as shown below.

```
=== Menu Achievement ===
1. Add achievement
2. View all achievements
3. Update achievement entry
4. Delete achievement entry
5. Back to main menu
```

17. If the user selects "Add Achievement," they can add their achievement and description, and the program will display a notification that the achievement has been successfully added.

```
=== Menu Achievement ===
1. Add achievement
2. View all achievements
3. Update achievement entry
4. Delete achievement entry
5. Back to main menu
Choice: 1
Title Achievement: first place in the running competition
Description: first place in the running competition at my school
Achievement added successfully!
```

PROGRAM OUTPUT

18. If the user selects "View all achievement," the program will display a list of achievement the user has recorded, including entry numbers, description and recording times.

```
=== Menu Achievement ===
1. Add achievement
2. View all achievements
3. Update achievement entry
4. Delete achievement entry
5. Back to main menu
Choice: 2
No: 1, Title: Menyelesaikan Project Python, Description: Berhasil bikin aplikasi CRUD digital diary dengan OOP Python, Date: 2025-09-20 16:25:51
No: 2, Title: Juara 1 Lomba Coding, Description: Lomba tingkat fakultas kategori web development, Date: 2025-09-20 16:25:51
No: 8, Title: first place in the running competition, Description: first place in the running competition at my school, Date: 2025-09-20 16:25:51
```

19. If user selects "Update achievement entry" the user can update their achievement based on ID, and then the program displays a notification that the changes have been successfully saved.

```
=== Menu Achievement ===
1. Add achievement
2. View all achievements
3. Update achievement entry
4. Delete achievement entry
5. Back to main menu
Choice: 3
Achievement ID to update: 2
New title: winner competition
New description: singing competition
Achievement updated successfully!
```

20. If the user chooses "Delete achievement entry", they can delete the achievement based on ID. The program then displays the list of remaining achievements after the deletion is successful.

```
=== Menu Achievement ===
1. Add achievement
2. View all achievements
3. Update achievement entry
4. Delete achievement entry
5. Back to main menu
Choice: 4
Achievement ID to delete: 1
Diary entry deleted successfully!

=== Remaining achievement ===
NO: 17, Activities: dawdssdawdadaw, DateTime: 2025-09-24 14:44:21
NO: 22, Activities: hiii, today was fun, DateTime: 2025-09-28 10:15:01
```

PROGRAM OUTPUT

21. If the user selects "Back to main menu", the program will return to main menu

```
=== Menu Achievement ===
1. Add achievement
2. View all achievements
3. Update achievement entry
4. Delete achievement entry
5. Back to main menu
Choice: 5

=== Menu Digital Diary's ===
1. Diary
2. Mood
3. Achievement
4. Logout
Choice Menu: 4
Goodbye, have a nice day!
```

22. If user selects "Logout", the program will display "Goodbye, have a nice day!"

```
=== Menu Achievement ===
1. Add achievement
2. View all achievements
3. Update achievement entry
4. Delete achievement entry
5. Back to main menu
Choice: 5

=== Menu Digital Diary's ===
1. Diary
2. Mood
3. Achievement
4. Logout
Choice Menu: 4
Goodbye, have a nice day!
```

PROGRAM OUTPUT

23. If user chooses exit, the program will end as the connection between python and MySQL will be terminated, and the program displayed the message "Goodbye!."

```
=== Welcome to MOODIE - your mood & digital diary! ===  
1. Register User  
2. Login  
3. Forgot Password  
4. Exit  
Enter your choice: 4  
Goodbye!
```

REQUIREMENTS

Hardware:

1. ACER Laptop

Operating System:

1. Windows 11 64-bit
2. Python

Software:

1. PyCharm
2. MySQL (XAMPP)
3. Ms. Word